

Session 1 Agenda

1. The problem: code tangled with "extra stuff"
2. The Proxy Design Pattern (the foundation everything else builds on)
3. What is AOP, really?
- 4.
6. Applying AOP using JDK Dynamic Proxy
5. The AOP family: AspectJ vs Spring AOP — clearing up the confusion
6. Spring's classic advice API (interfaces, not annotations yet)
7. **Lab 2** — implement advice types through interfaces

Note: we're using **plain Spring Core** this session — no Spring Boot yet.
Annotations like `@Aspect` and `@Before` are coming in **Session 2**.

A Simple Banking Service

```
public class AccountService {  
  
    public void withdraw(String accountId, double amount) {  
        // business logic  
        Account acc = repository.find(accountId);  
        acc.setBalance(acc.getBalance() - amount);  
        repository.save(acc);  
    }  
  
    public void deposit(String accountId, double amount) {  
        Account acc = repository.find(accountId);  
        acc.setBalance(acc.getBalance() + amount);  
        repository.save(acc);  
    }  
}
```

Clean. Simple. Does one thing.

Now Add "Real World" Requirements

- Log every call (who, when, with what arguments)

- Check the user is authorized
- Measure how long it took
- Wrap it in a transaction
- Catch and log exceptions

Where does all this code go?

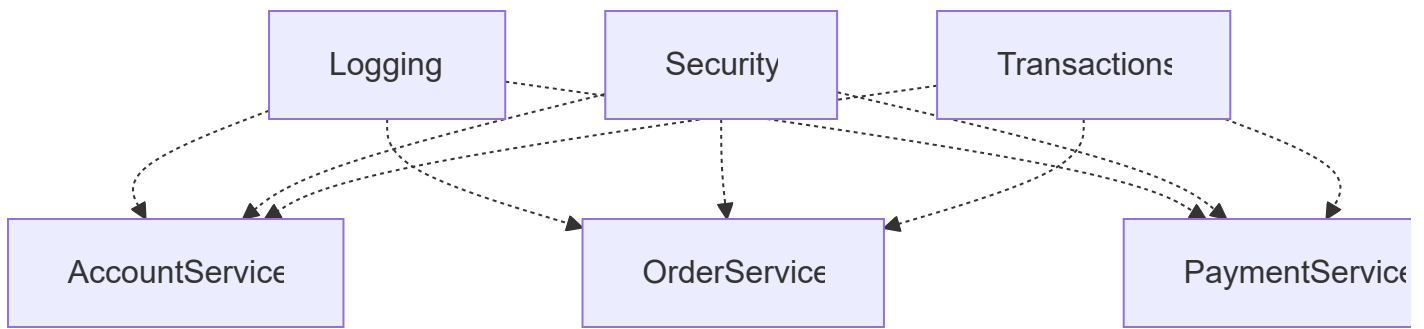
Option A: Stuff It Into Every Method 🤖

```
public void withdraw(String accountId, double amount) {
    long start = System.currentTimeMillis();
    log.info("withdraw called with {} {}", accountId, amount);
    if (!security.isAuthenticated(accountId)) {
        throw new SecurityException("not allowed");
    }
    try {
        Account acc = repository.find(accountId);
        acc.setBalance(acc.getBalance() - amount);
        repository.save(acc);
    } catch (Exception e) {
        log.error("withdraw failed", e);
        throw e;
    } finally {
        log.info("withdraw took {} ms", System.currentTimeMillis() - start);
    }
}
```

Now imagine repeating this in **every** method, of **every** service, in the **whole app**.

This Has a Name: Cross-Cutting Concerns

Concerns like **logging**, **security**, **transactions**, **caching**, **error-handling** don't belong to *one* class — they "cut across" many classes at once.



Problems: **code duplication, mixed responsibilities, hard to maintain**
(want to change your logging format? Now you touch 50 files).

The Question

What if we could write the business logic **once**, cleanly,
and "attach" the logging/security/timing **from the outside**,
without touching the business class at all?

This is exactly what **AOP** solves.

But to understand *how* Spring pulls this off, we first need one classic design pattern: the **Proxy**.

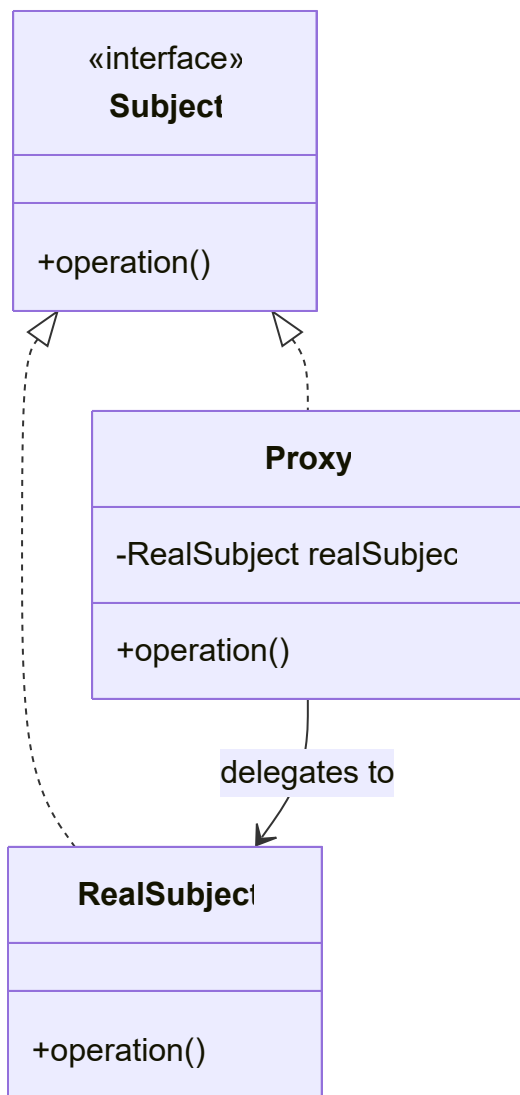
The Proxy Design Pattern

Definition: A Proxy is an object that stands *in front of* another object (the "real" object) and controls access to it — you call the proxy, the proxy decides what to do, then (usually) forwards the call to the real object.

Real-life analogies:

- A **credit card** is a proxy for your bank account (checks funds, logs the transaction, then moves the money)
 - A **receptionist** is a proxy for a busy executive (filters, logs, and forwards visitors)
 - A **security guard** at a building entrance is a proxy for the building itself
-

Proxy Pattern — The Shape of It



Both `RealSubject` and `Proxy` implement the same interface, so the **caller can't tell the difference** — it just calls `operation()` .

Proxy Pattern — Plain Java Code

```
// The contract both real object and proxy follow
interface AccountService {
    void withdraw(String accountId, double amount);
}

// The REAL object - pure business logic, nothing else
class AccountServiceImpl implements AccountService {
    public void withdraw(String accountId, double amount) {
        System.out.println("Withdrawing " + amount + " from " + accountId);
        // ... actual balance update logic
    }
}
```

```
}  
}
```

Proxy Pattern — The Proxy Class

```
class AccountServiceProxy implements AccountService {  
  
    private final AccountService realService; // the real object  
  
    public AccountServiceProxy(AccountService realService) {  
        this.realService = realService;  
    }  
  
    @Override  
    public void withdraw(String accountId, double amount) {  
        long start = System.currentTimeMillis();  
        System.out.println("LOG: withdraw called with " + accountId + ", " +  
amount);  
  
        realService.withdraw(accountId, amount); // <-- delegate to the real  
object  
  
        System.out.println("LOG: took " + (System.currentTimeMillis() - start)  
+ " ms");  
    }  
}
```

```
AccountService service = new AccountServiceProxy(new AccountServiceImpl());  
service.withdraw("acc-1", 100.0); // caller has NO idea a proxy is involved
```

The business class never changed. The "extra stuff" lives entirely in the proxy.

Two Flavors of Proxy

Type	How it's made	Example
Static proxy	You hand-write a proxy class (like we just did)	AccountServiceProxy above

Type	How it's made	Example
Dynamic proxy	The proxy class is generated at runtime by code	JDK Proxy class, CGLIB

Hand-writing a proxy for every class is exactly the duplication problem again.

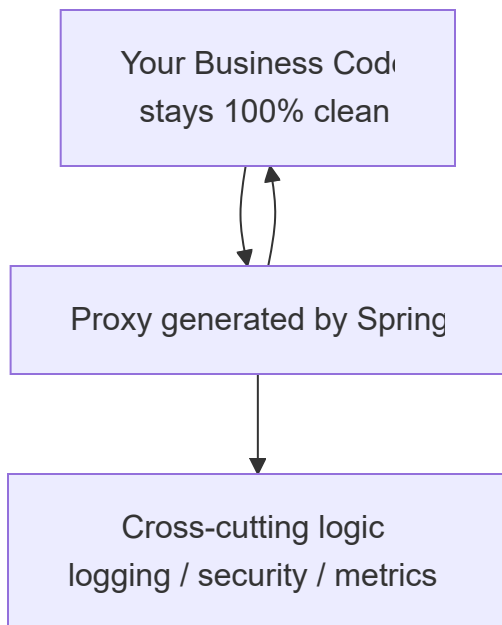
Dynamic proxies let a framework generate the proxy for us, automatically.

This is precisely what **Spring AOP does under the hood**.

Now: What Is AOP?

AOP = Aspect-Oriented Programming

A programming paradigm that lets you **separate cross-cutting concerns** (logging, security, transactions...) from your core business logic, and re-attach them **declaratively**, without modifying the business class.

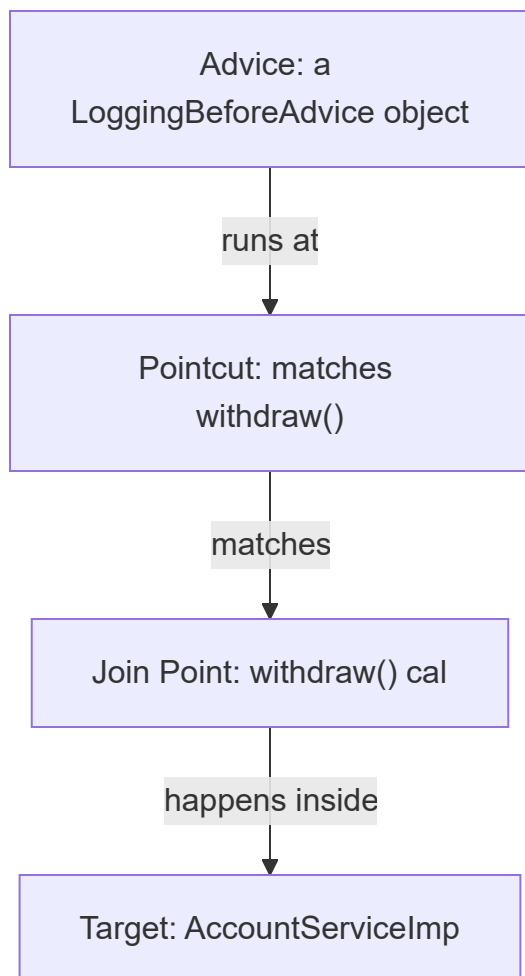


Under the hood: **Spring AOP = automatically generated dynamic proxies**, configured by plugging in advice objects instead of hand-writing proxy classes.

AOP Vocabulary — Don't Skip This Slide

Term	Meaning
Aspect	The module containing the cross-cutting logic (where + what)
Join Point	A point during execution where behavior <i>could</i> be applied (e.g. a method call)
Advice	The actual code that runs at a join point (the "what to do")
Pointcut	An expression/object that says <i>which</i> join points the advice applies to (the "where")
Weaving	The process of linking aspects with target objects (creating the proxy)
Target Object	The object being advised (your real business object)

Putting the Vocabulary Together



In plain English:

"I have an **Advice** object (`LoggingBeforeAdvice`) that should run **before**

any method matched by my **Pointcut** — and that code will fire at the moment `withdraw()` is actually called (join point) on the real `AccountServiceImpl` object (target)."

We'll formalize "Advice" and "Pointcut" as real Java types in a moment.

Java's Built-in Dynamic Proxy (JDK Proxy)

Works **only with interfaces**. Uses `InvocationHandler`:

```
import java.lang.reflect.*;

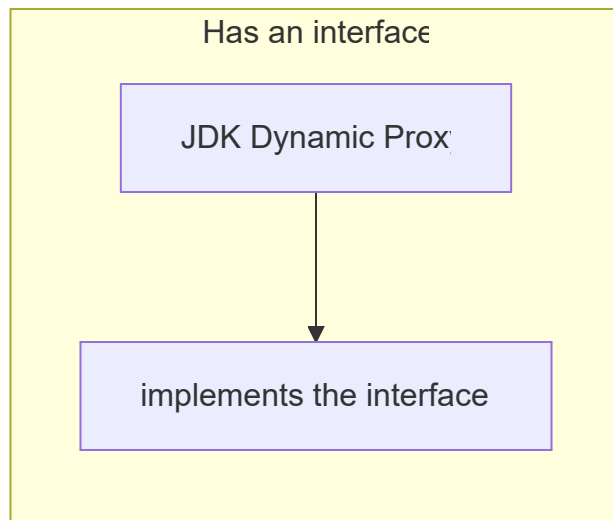
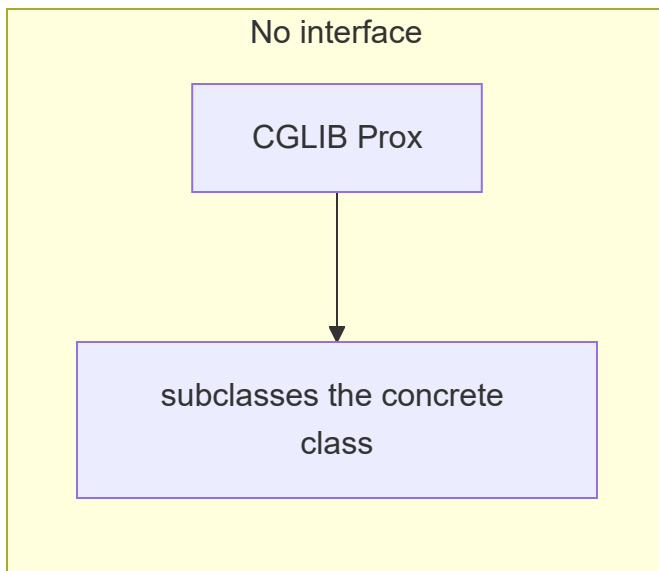
class LoggingHandler implements InvocationHandler {
    private final Object target;
    LoggingHandler(Object target) { this.target = target; }

    public Object invoke(Object proxy, Method method, Object[] args) throws
    Throwable {
        System.out.println("LOG: calling " + method.getName());
        Object result = method.invoke(target, args); // call the real method
        System.out.println("LOG: finished " + method.getName());
        return result;
    }
}
```

```
AccountService real = new AccountServiceImpl();
AccountService proxy = (AccountService) Proxy.newProxyInstance(
    AccountService.class.getClassLoader(),
    new Class[]{AccountService.class},
    new LoggingHandler(real)
);
proxy.withdraw("acc-1", 50.0); // goes through LoggingHandler first!
```

What About Classes Without an Interface?

JDK dynamic proxy needs an interface. If your class doesn't implement one, Spring falls back to **CGLIB** — it generates a **subclass** of your class at runtime and overrides its methods to inject the extra behavior.



You don't have to write either of these yourself — Spring does it for you (we'll get there later this session). But first, let's build one ourselves by hand, so the "magic" is gone before we ever meet Spring AOP.

Lab 1 — Build Your Own JDK Dynamic Proxy

Setup (plain Java, no Spring needed for this lab):

1. Define an interface `NotificationService` with `sendEmail(String to, String message)` and `sendSms(String to, String message)`
2. Implement it with `NotificationServiceImpl` — just print what it "sent"

Tasks:

1. Write an `InvocationHandler` called `LoggingHandler` that logs the method name and arguments **before** calling the real method, and the return value **after**
 2. Use `Proxy.newProxyInstance(...)` to wrap `NotificationServiceImpl` and call both methods through the proxy — confirm the logging wraps each call
 3. Add timing inside the same handler: measure and print how long each call took
-

The AOP Family — Where Does "Spring AOP" Fit?

You will constantly hear two names in the same breath: **AspectJ** and **Spring AOP**. They are related, but they are **not the same thing** — and mixing them up is the #1 source of confusion for people learning this.

AOP itself is just a *paradigm* — a set of ideas (aspect, join point, weaving...). Multiple frameworks implement those ideas differently. Let's meet the two that matter to us.

AspectJ — The Original, Full-Powered Implementation

- Created in the late 1990s (Xerox PARC, later an Eclipse project) as an actual **extension of the Java language**, with its **own compiler** (`ajc`)
 - Because it extends the language itself, it can weave into **any** join point: public/private/protected method calls, field reads/writes, object construction (`new SomeClass()`), static initializers, exception handlers
 - Two ways to weave:
 - **Compile-Time Weaving (CTW)**: compile with `ajc` instead of `javac` — the cross-cutting code is baked directly into the `.class` bytecode
 - **Load-Time Weaving (LTW)**: a Java agent rewrites bytecode as classes are loaded by the JVM — no special compiler needed
 - Trade-off: very powerful, but it adds real build complexity — a special compiler or a Java agent, extra configuration
-

Spring AOP — A Simpler, Proxy-Based Implementation

- Spring's own built-in AOP module (`spring-aop`) — designed to bring AOP's biggest wins to everyday Spring apps **without needing a special compiler or agent**

- **How it weaves:** at **runtime**, via dynamic proxies — exactly the JDK Proxy / CGLIB mechanism you just used by hand in Lab 1
 - **The restriction:** it can only advise **method calls on objects that Spring itself creates and manages as beans** — no advising private methods, constructors, static methods, field access, or a plain `new SomeService()` created outside Spring's control
 - This restriction is intentional: it keeps setup trivial — zero extra build steps — and it covers the vast majority of real use cases (logging, security, transactions, caching)
-

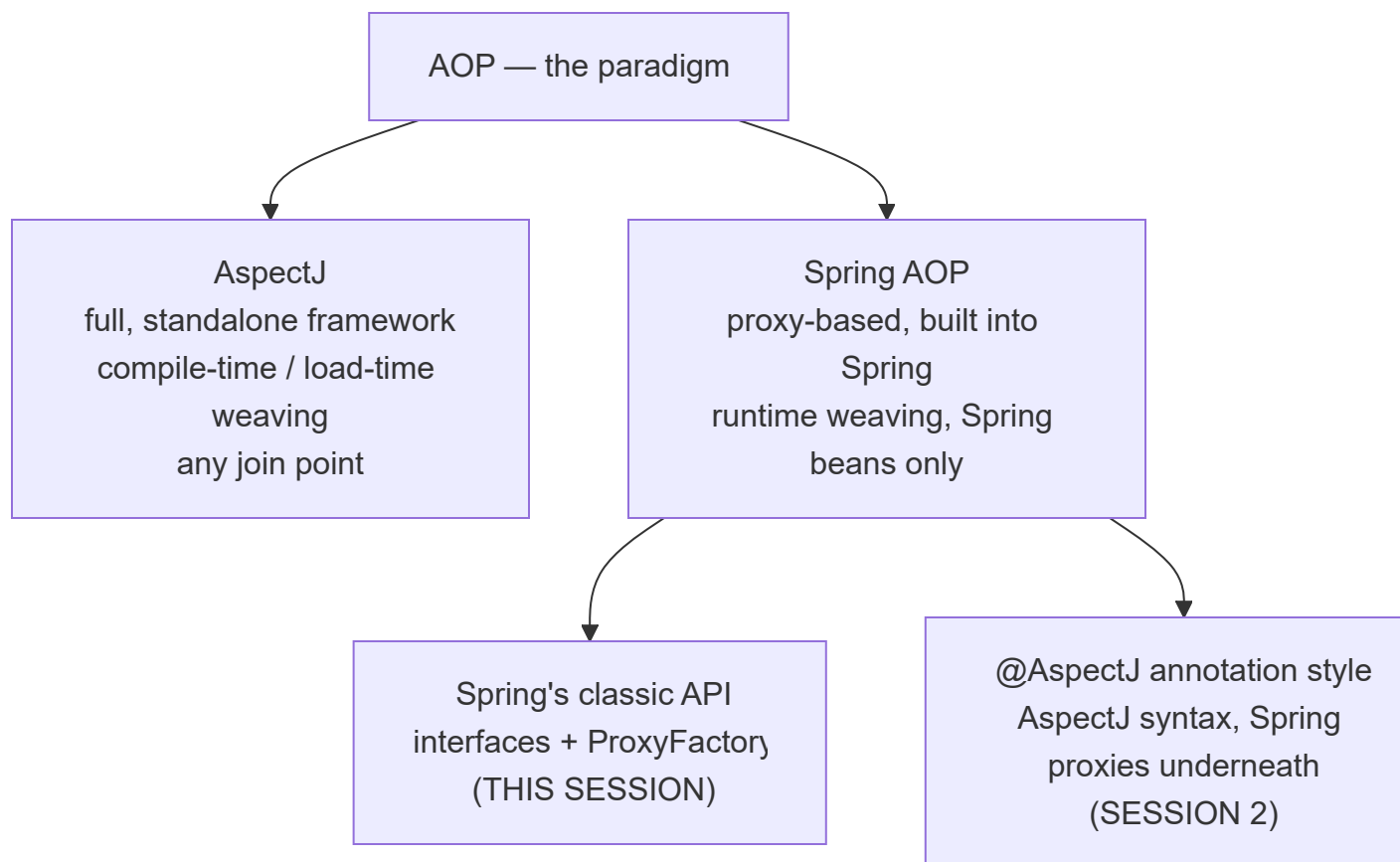
So Why Do Spring Docs Keep Mentioning AspectJ?

Two areas of overlap cause most of the confusion:

1. **Pointcut expression language** — Spring AOP *reuses* AspectJ's well-designed syntax for describing "where" advice applies (`execution(...)`, `within(...)`, etc.) as a convenient DSL — but Spring parses and matches these expressions **itself**; it still just builds a proxy. The real AspectJ weaver is never invoked. (*We'll use this syntax in Session 2.*)
2. **@Aspect / @Before / @Around annotations** — these annotation *types* actually live in the AspectJ library (`aspectjweaver.jar` ships them), but Spring just reads them as configuration hints for building a proxy — again, no compile-time or load-time weaving happens.

Bottom line: seeing AspectJ-flavored syntax in a Spring app almost always still means **Spring AOP proxies underneath**, not real AspectJ weaving.

The Full Picture



Session	What you'll write	Weaving under the hood
Session 1 (today)	Advice interfaces + ProxyFactory	Spring AOP (runtime proxy)
Session 2	@Aspect , @Before , execution(...)	Still Spring AOP (runtime proxy) — nicer syntax

Today we build the mental model with the classic API. In Session 2, the annotations will feel like a shortcut for something you already understand — not new magic.

Setting Up Spring AOP (Spring Core, No Boot)

Maven — just two artifacts, no Spring Boot starter:

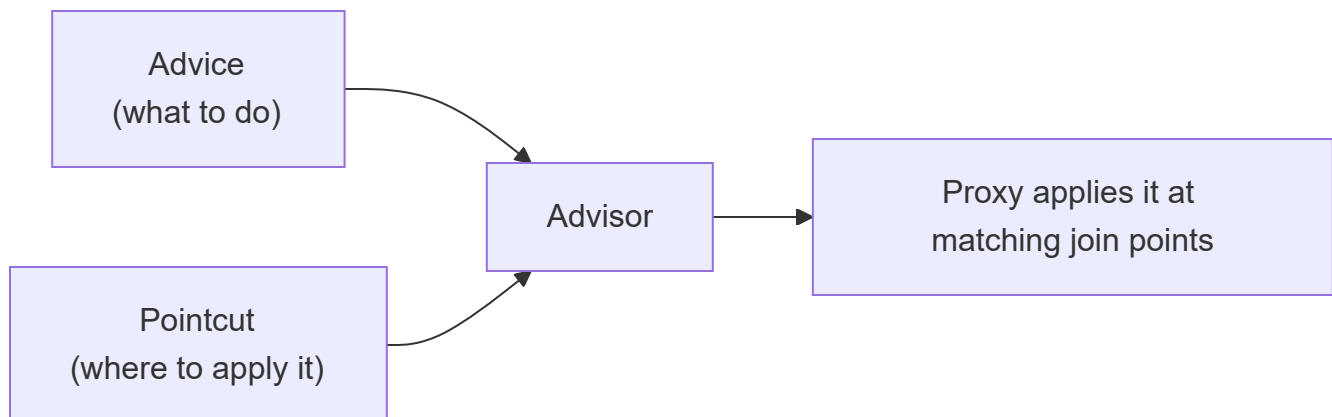
```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>6.1.0</version>
</dependency>
<dependency>
```

```
<groupId>org.springframework</groupId>
<artifactId>spring-aop</artifactId>
<version>6.1.0</version>
</dependency>
```

spring-aop already ships everything we need today: the advice interfaces, ProxyFactory / ProxyFactoryBean, and the AOP Alliance MethodInterceptor contract. No annotations, no auto-configuration — we wire everything ourselves, so you see exactly what's happening.

Two Building Blocks: Advice + Pointcut = Advisor

- **Advice** — *what* to do (the interfaces we're about to meet)
- **Pointcut** — *where* to do it (which methods); the simplest pointcut matches **everything**
- **Advisor** — an Advice + a Pointcut bundled together: *"apply THIS behavior at THESE join points"*



If you don't need to be selective, you can skip the Pointcut entirely and just attach Advice directly to a proxy — it will apply to **every** method. We'll do that first, then add Pointcuts once we need to be selective.

MethodBeforeAdvice — Runs Before the Method

```
public class LoggingBeforeAdvice implements MethodBeforeAdvice {

    @Override
    public void before(Method method, Object[] args, Object target) throws
```

```

Throwable {
    System.out.println("BEFORE: " + method.getName() +
        " args=" + Arrays.toString(args));
}
}

```

Typical uses: logging, authorization checks, input validation.

Notice: no `JoinPoint` object here — Spring hands you the raw `Method`, the arguments, and the target object directly.

AfterReturningAdvice — Runs After Success

```

public class LoggingAfterReturningAdvice implements AfterReturningAdvice {

    @Override
    public void afterReturning(Object returnValue, Method method,
        Object[] args, Object target) throws Throwable
    {
        System.out.println("SUCCESS: " + method.getName() +
            " returned " + returnValue);
    }
}

```

Only runs if the method completed **without throwing**. You get direct access to the return value.

ThrowsAdvice — Runs After an Exception

`ThrowsAdvice` is a **marker interface** — it has no method to override. Instead, Spring finds a method named `afterThrowing` on your class via reflection, matching one of a few allowed signatures:

```

public class LoggingThrowsAdvice implements ThrowsAdvice {

    // Simplest form: just the exception
    public void afterThrowing(Exception ex) {
        System.out.println("ERROR: " + ex.getMessage());
    }
}

```

```
// Or the more detailed form, if you need method/args/target too:
public void afterThrowing(Method method, Object[] args, Object target,
Exception ex) {
    System.out.println("ERROR in " + method.getName() + ": " +
ex.getMessage());
}
}
```

You can even declare multiple `afterThrowing` overloads for different exception types — Spring picks the most specific match.

MethodInterceptor — Around Advice

The most powerful one. Wraps the **entire** call — you decide if/when it runs, and can change arguments or the return value.

```
public class TimingInterceptor implements MethodInterceptor {

    @Override
    public Object invoke(MethodInvocation invocation) throws Throwable {
        long start = System.currentTimeMillis();
        System.out.println("START: " + invocation.getMethod().getName());

        Object result = invocation.proceed(); // <-- calls the real method!

        System.out.println("END: took " +
            (System.currentTimeMillis() - start) + "ms");
        return result;
    }
}
```

⚠ Forgetting `invocation.proceed()` means the **real method never runs** — exactly like forgetting `pjp.proceed()` would, if you'd seen that name before.

Important: `MethodInterceptor` isn't Spring-specific — it comes from the **AOP Alliance**, a shared standard interface used across multiple Java AOP frameworks. Spring simply implements it.

No Dedicated "Runs No Matter What" Interface

Unlike what you might expect, classic Spring AOP has **no separate interface** for "run after the method, whether it succeeded or threw" (the `finally`-style behavior). You get it naturally from `MethodInterceptor`:

```
public Object invoke(MethodInvocation invocation) throws Throwable {
    try {
        return invocation.proceed();
    } finally {
        System.out.println("ALWAYS RUNS - success or exception");
    }
}
```

Building the Proxy — ProxyFactory

```
AccountService target = new AccountServiceImpl();

ProxyFactory factory = new ProxyFactory(target);
factory.addAdvice(new LoggingBeforeAdvice());
factory.addAdvice(new LoggingAfterReturningAdvice());
factory.addAdvice(new TimingInterceptor());

AccountService proxy = (AccountService) factory.getProxy();
proxy.withdraw("acc-1", 100.0);
```

Compare this to Lab 1: same idea as `Proxy.newProxyInstance` + `InvocationHandler`, but Spring handles the reflection plumbing for you — you just plug in Advice objects. The order you call `addAdvice(...)` matters, just like the handler-chaining you did in Lab 1.

Being Selective — Pointcuts

So far every advice applies to **every** method. To apply advice only to specific methods, pair it with a `Pointcut` inside an `Advisor`:

```
NameMatchMethodPointcut pointcut = new NameMatchMethodPointcut();
pointcut.setMappedName("withdraw"); // only applies to withdraw()
// pointcut.setMappedNames("with*", "get*"); // wildcards also work

Advisor advisor = new DefaultPointcutAdvisor(pointcut, new
```

```
LoggingBeforeAdvice();

ProxyFactory factory = new ProxyFactory(target);
factory.addAdvisor(advisor);

AccountService proxy = (AccountService) factory.getProxy();
```

NameMatchMethodPointcut is the classic, pre-AspectJ-expression way of saying "where." The richer `execution(...)` expression syntax is coming in Session 2 — same underlying idea, nicer notation.

Wiring It Into the Spring Container — ProxyFactoryBean

We're using plain Spring Core, so here's the container-managed version — so other beans can have this proxy `@Autowired` into them, the way you already wire beans together:

```
@Configuration
public class AppConfig {

    @Bean
    public AccountServiceImpl accountServiceTarget() {
        return new AccountServiceImpl();
    }

    @Bean
    public MethodBeforeAdvice loggingBeforeAdvice() {
        return new LoggingBeforeAdvice();
    }

    @Bean
    public ProxyFactoryBean accountService() {
        ProxyFactoryBean factory = new ProxyFactoryBean();
        factory.setTarget(accountServiceTarget());
        factory.setInterceptorNames("loggingBeforeAdvice");
        return factory;
    }
}
```

ProxyFactoryBean is the classic, container-managed way to build AOP proxies — before annotation-driven auto-proxying existed. It's a little

verbose; that verbosity is exactly *why* Spring later introduced the

`@AspectJ` annotation style you'll meet in Session 2. Same mechanism underneath.

Session 1 Recap

- **Proxy pattern**: wrap a real object to add behavior without changing it — you built one by hand in Lab 1
 - **Dynamic proxies** generate that wrapper automatically (JDK Proxy / CGLIB)
 - **AOP complements OOP** — it handles concerns that cut across many unrelated classes
 - **AspectJ** = full, standalone AOP framework (compile-time/load-time weaving, any join point)
 - **Spring AOP** = simpler, proxy-based subset — only advises Spring bean method calls
 - Spring's **classic advice interfaces**: `MethodBeforeAdvice`, `AfterReturningAdvice`, `ThrowsAdvice`, `MethodInterceptor`
 - `Advice + Pointcut = Advisor`; `ProxyFactory` / `ProxyFactoryBean` build the proxy
-

Lab 2 — Implement Advice Types Through Interfaces

Setup:

1. Add `spring-context` and `spring-aop` to your project (no Spring Boot)
2. Create an `InventoryService` interface with `checkStock(String sku)` (returns an `int`) and `reserveStock(String sku, int qty)` — throw `IllegalStateException` if `qty > 100`
(to test exception advice)
3. Implement it with `InventoryServiceImpl`

Tasks:

1. Implement `MethodBeforeAdvice` to log every method call (name + arguments)
2. Implement `AfterReturningAdvice` to log the return value of `checkStock`
3. Implement `ThrowsAdvice` to log any exception thrown by `reserveStock`
4. Implement `MethodInterceptor` (Around) that times every call **and** achieves "runs no matter what" behavior using `try/finally`
5. Use `ProxyFactory` to build a proxy of `InventoryServiceImpl` with **all four advices attached** — verify each fires at the right moment, and in the right order
6. Wire the same setup into a Spring `ApplicationContext` using `@Configuration` + `ProxyFactoryBean`, so the proxy is a container-managed bean you could `@Autowired`

elsewhere

Bonus: Use a `NameMatchMethodPointcut` so your logging `MethodBeforeAdvice` only applies to `reserveStock`, not `checkStock` — confirm `checkStock` calls are no longer logged.